Predicting MLB Team Success

AUTHOR
Sean Tessman

1 Project Goal

- Use data to understand what makes MLB teams successful
- Focus on team batting statistics (2024 season)

1.1 Questions:

1. What stats lead to winning teams?
2. Can teams be grouped into offensive types?
3. Do certain styles win more?

---

2 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, confusion_matrix

plt.rcParams["figure.figsize"] = (8, 5)
plt.rcParams["figure.dpi"] = 120
```

---

3 Load the Data

```
batting = pd.read_csv("Data/mlb_team_batting_2024.csv")
standings = pd.read_csv("Data/mlb-standings-2024.csv")

print("Batting shape:", batting.shape)
print("Standings shape:", standings.shape)
batting.head()
```

```
Batting shape: (32, 29)
Standings shape: (30, 26)
```

	Tm	#Bat	BatAge	R/G	G	PA	AB	R	H	2B	...	SLG	OPS	OPS+	TB	GDP	HBP	SH	SF
0	Arizona Diamondbacks	51	28.6	5.47	162	6284	5522	886	1452	271	...	0.440	0.777	115	2430	112	84	34	66
1	Atlanta Braves	55	29.3	4.35	162	6075	5481	704	1333	273	...	0.415	0.724	100	2275	119	58	9	39
2	Baltimore Orioles	60	26.9	4.85	162	6176	5567	786	1391	262	...	0.435	0.751	116	2424	71	64	6	45
3	Boston Red Sox	57	27.3	4.64	162	6192	5577	751	1404	311	...	0.423	0.741	106	2357	110	73	7	40
4	Chicago Cubs	56	27.8	4.54	162	6116	5441	736	1318	253	...	0.393	0.710	100	2139	89	70	17	41

5 rows × 29 columns

4 Preview the Standings Data

```
standings.head()
```

	Rk	Tm	League	is_NL	W	L	W-L%	R	RA	Rdiff	...	vWest	Inter	Home	Road	ExInn	1Run	vRHP	vLH
0	1	Los Angeles Dodgers	NL	1	98	64	0.605	5.2	4.2	1.0	...	31-21	30-16	52-29	46-35	7-Sep	21-17	62-47	36-17
1	2	Philadelphia Phillies	NL	1	95	67	0.586	4.8	4.1	0.7	...	22-10	26-20	54-27	41-40	8-Aug	23-20	61-43	34-24
2	3	New York Yankees	AL	0	94	68	0.580	5.0	4.1	0.9	...	21-12	23-23	44-37	50-31	8-Aug	19-18	73-45	21-23
3	4	Milwaukee Brewers	NL	1	93	69	0.574	4.8	4.0	0.8	...	15-19	31-15	47-34	46-35	9-Jun	28-25	69-45	24-24
4	5	San Diego Padres	NL	1	93	69	0.574	4.7	4.1	0.6	...	27-25	27-19	45-36	48-33	2-Oct	22-19	66-50	27-19

5 rows × 26 columns

5 Clean and Merge the Data

```
batting = batting[~batting["Tm"].isin(["LgAvg per 600 PA", "MLB Total"])].copy()

batting = batting[["Tm", "R", "HR", "BA", "RBI", "SB", "OBP", "SLG", "OPS", "BB", "SO"]].copy()
```

```
standings = standings.rename(columns={
    "Team": "Tm",
    "W-L%": "Win%",
    "W-L% ": "Win%",
    "Pct": "Win%",
    "WPct": "Win%",
    "Lg": "League",
    "tm": "Tm",
    "team": "Tm",
    "league": "League"
})

standings = standings[["Tm", "Win%", "League"]].copy()

def clean_team_name(name):
    name = str(name).strip()
    name = name.replace("Arizona Diamondbacks", "Arizona D'Backs")
    return name

batting["Tm"] = batting["Tm"].apply(clean_team_name)
standings["Tm"] = standings["Tm"].apply(clean_team_name)

mlb = batting.merge(standings, on="Tm", how="inner")

mlb["winning_team"] = (mlb["Win%"] > 0.500).astype(int)

print("Merged dataset shape:", mlb.shape)
mlb.head()
```

```
Merged dataset shape: (30, 14)
```

	Tm	R	HR	BA	RBI	SB	OBP	SLG	OPS	BB	SO	Win%	League	winning_team
0	Arizona D'Backs	886	211	0.263	845	119	0.337	0.440	0.777	569	1265	0.549	NL	1
1	Atlanta Braves	704	213	0.243	674	69	0.309	0.415	0.724	485	1461	0.549	NL	1
2	Baltimore Orioles	786	235	0.250	759	98	0.315	0.435	0.751	489	1359	0.562	AL	1
3	Boston Red Sox	751	194	0.252	724	144	0.319	0.423	0.741	493	1570	0.500	AL	0
4	Chicago Cubs	736	170	0.242	696	143	0.317	0.393	0.710	546	1362	0.512	NL	1

What this does:

- Removes non-team rows

- Keeps the key batting stats

- Merges batting and standings into one clean dataset

- Creates `winning_team`, where 1 = above .500 and 0 = not above .500

---

6 Check the Final Dataset

```
mlb.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   Tm            30 non-null     object 
 1   R             30 non-null     int64  
 2   HR            30 non-null     int64  
 3   BA            30 non-null     float64
 4   RBI           30 non-null     int64  
 5   SB            30 non-null     int64  
 6   OBP           30 non-null     float64
 7   SLG           30 non-null     float64
 8   OPS           30 non-null     float64
 9   BB            30 non-null     int64  
 10  SO            30 non-null     int64  
 11  Win%          30 non-null     float64
 12  League        30 non-null     object 
 13  winning_team  30 non-null     int64  
dtypes: float64(5), int64(7), object(2)
memory usage: 3.4+ KB
```

What this shows:

- 30 teams remain after cleaning

- No missing values in the main variables

- The data is ready for analysis

---

7 Key Variables

- `R` = Runs
- `HR` = Home Runs
- `BA` = Batting Average
- `RBI` = Runs Batted In
- `SB` = Stolen Bases
- `OBP` = On-Base Percentage
- `SLG` = Slugging Percentage
- `OPS` = On-Base Plus Slugging
- `Win%` = Winning Percentage
- `winning_team` = 1 if above .500, 0 otherwise

8 Summary Statistics

```
summary_stats = mlb[["R", "HR", "BA", "RBI", "SB", "OBP", "SLG", "OPS", "Win%"]].describe().T
summary_stats
```

	count	mean	std	min	25%	50%	75%	max
R	30.0	711.433333	75.465308	507.000	671.25000	701.5000	757.75000	886.000
HR	30.0	181.766667	26.940142	133.000	165.00000	178.0000	195.50000	237.000
BA	30.0	0.243167	0.011014	0.221	0.23500	0.2430	0.24800	0.263
RBI	30.0	679.300000	74.140663	485.000	640.50000	666.5000	725.50000	845.000
SB	30.0	120.566667	43.087468	65.000	90.25000	112.5000	142.25000	223.000
OBP	30.0	0.311867	0.012517	0.278	0.30425	0.3100	0.31900	0.337
SLG	30.0	0.398967	0.024602	0.340	0.38125	0.3955	0.41725	0.446
OPS	30.0	0.711033	0.035847	0.618	0.68625	0.7035	0.73925	0.781
Win%	30.0	0.499967	0.077075	0.253	0.47050	0.5120	0.54900	0.605

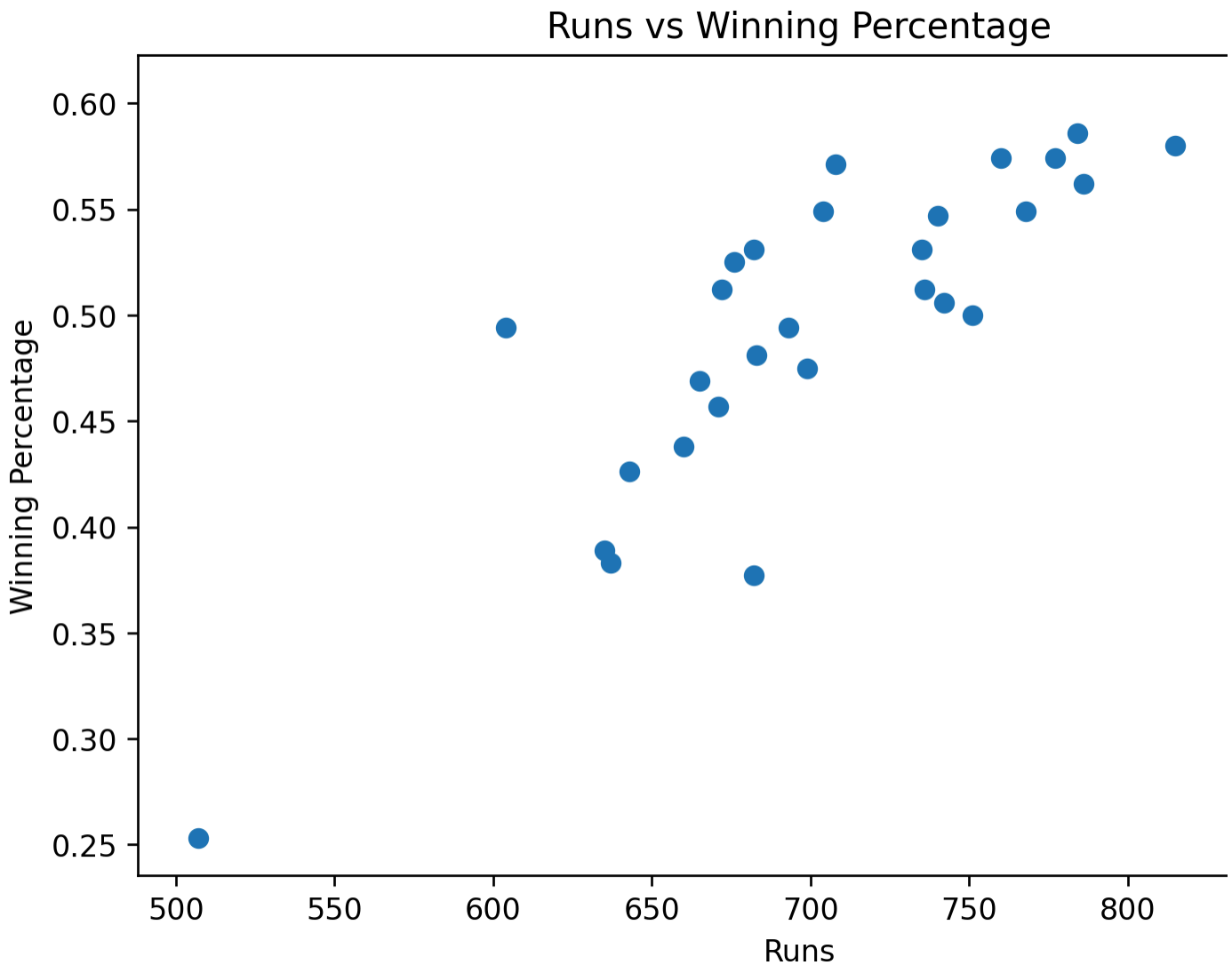
Main insight:

- There is noticeable variation across teams in runs, OPS, and winning percentage

- That gives us enough spread to model team success

9 Runs vs Winning Percentage

```
plt.scatter(mlb["R"], mlb["Win%"])
plt.xlabel("Runs")
plt.ylabel("Winning Percentage")
plt.title("Runs vs Winning Percentage")
plt.show()
```



Takeaway:

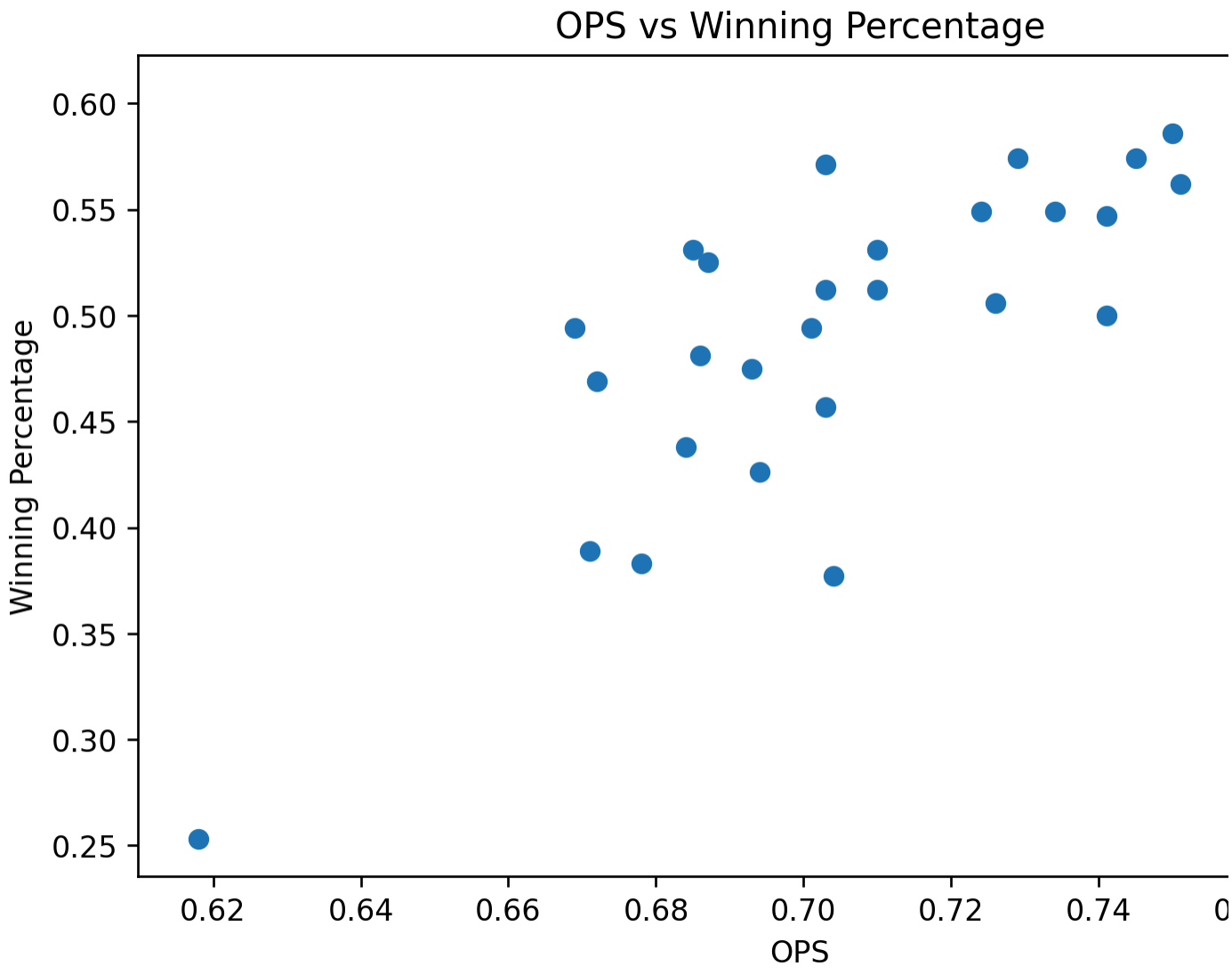
- There is a clear upward trend

- Teams that scored more runs usually had better records

---

10 OPS vs Winning Percentage

```
plt.scatter(mlb["OPS"], mlb["Win%"])
plt.xlabel("OPS")
plt.ylabel("Winning Percentage")
plt.title("OPS vs Winning Percentage")
plt.show()
```



Takeaway:

- Teams with higher OPS were generally more successful

- OPS looks like one of the strongest offensive measures

---

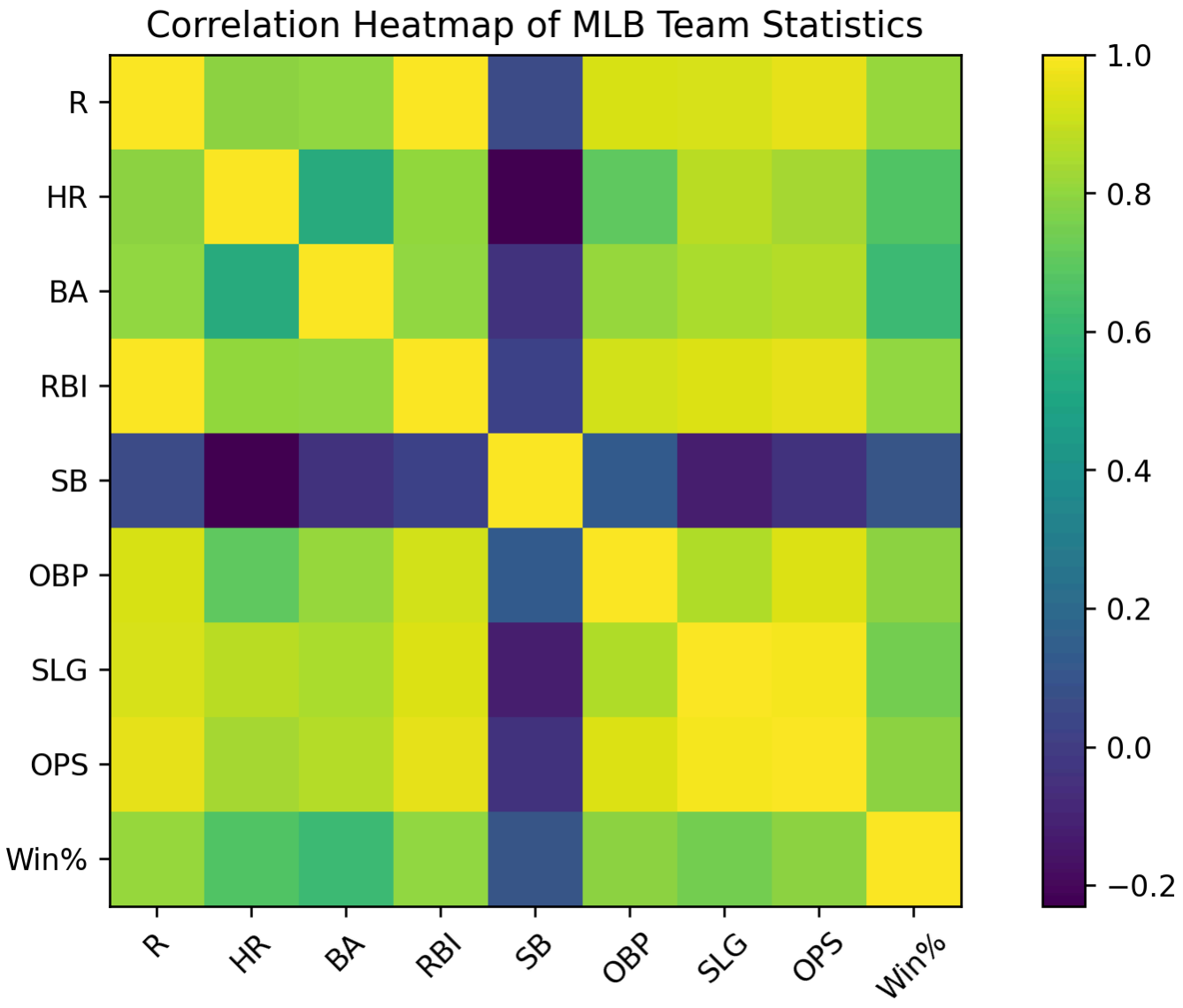
11 Correlation Matrix

```
corr_matrix = mlb[["R", "HR", "BA", "RBI", "SB", "OBP", "SLG", "OPS", "Win%"]].corr()
plt.figure()
plt.imshow(corr_matrix)

plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation=45)
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)

plt.colorbar()
```

```
plt.title("Correlation Heatmap of MLB Team Statistics")
plt.tight_layout()
plt.show()
```



Key findings:

- Runs and Win% have a strong positive relationship

- OPS and Win% are also strongly related

- OBP and SLG are both highly connected to team success

---

12 Logistic Regression Goal

- Predict whether a team finished **above .500**
- Use batting variables only

- This is a binary classification problem, so logistic regression is a good fit

---

13 Fit Logistic Regression Model

```
X = mlb[["R", "HR", "BA", "RBI", "SB", "OBP", "SLG", "OPS"]].copy()
y = mlb["winning_team"]

log_model = LogisticRegression(max_iter=2000)
log_model.fit(X, y)

predictions = log_model.predict(X)
probabilities = log_model.predict_proba(X)[:, 1]

mlb["predicted_win_class"] = predictions
mlb["predicted_win_probability"] = probabilities

print("Training accuracy:", round(accuracy_score(y, predictions), 3))
print("Confusion matrix:")
print(confusion_matrix(y, predictions))
```

```
Training accuracy: 0.833
Confusion matrix:
[[11  2]
 [ 3 14]]
```

Main result:

- The model classifies teams fairly well using batting stats alone

- This shows offense is strongly tied to team success

---

14 Logistic Regression Coefficients

```
coef_table = pd.DataFrame({
    "Variable": X.columns,
    "Coefficient": log_model.coef_[0],
    "Odds_Ratio": np.exp(log_model.coef_[0])
}).sort_values("Coefficient", ascending=False)

coef_table
```

	Variable	Coefficient	Odds_Ratio
0	R	0.219625	1.245610
1	HR	0.003612	1.003618

	Variable	Coefficient	Odds_Ratio
5	OBP	-0.001306	0.998695
2	BA	-0.008020	0.992012
6	SLG	-0.015418	0.984700
7	OPS	-0.016717	0.983422
4	SB	-0.019582	0.980608
3	RBI	-0.174256	0.840082

What this shows:
This chunk pulls the coefficients from the logistic regression model to show how each offensive stat affects the probability of a team being a winning team. Positive coefficients increase the chance of winning, while negative ones either decrease it or reflect overlap with stronger variables in the model. The odds ratio is just a more interpretable version of the coefficient. So values above 1 mean the odds of winning go up as that stat increases, while values below 1 mean the odds go down or the effect is weaker.

Key takeaways:

- **Runs (R)** has the strongest positive impact on winning
    - Highest coefficient (0.2196) and odds ratio (1.2456)
    - Teams that score more runs are significantly more likely to win
- **Home Runs (HR)** has a small positive effect
    - Slight increase in winning probability, but much weaker than runs
- **RBI has a negative coefficient (-0.1743)**
    - Likely due to multicollinearity with runs (R and RBI measure similar things)
    - The model prioritizes runs over RBI when both are included
- **OPS, OBP, SLG, and BA all have small negative coefficients**
    - This does not mean they are bad for winning
    - It suggests overlap between these variables (they are highly correlated)
- **Stolen Bases (SB)** has a negative effect
    - Indicates speed alone is not a strong predictor of winning compared to hitting power

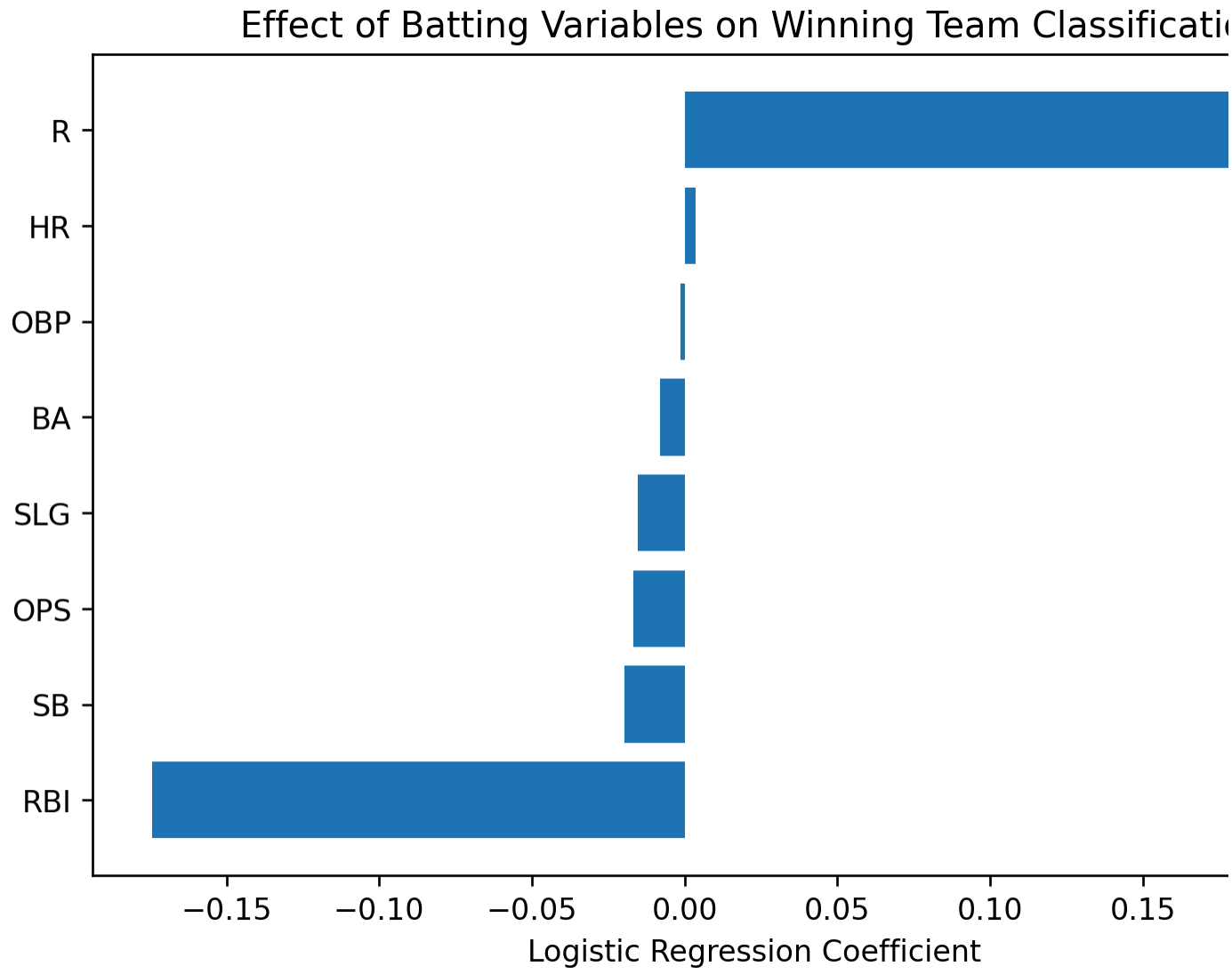
Runs scored is the single most important offensive factor in predicting team success. While other hitting stats are important, many of them capture similar information, which is why the model primarily relies on runs as the strongest predictor of winning.

---

15 Coefficient Plot

```
coef_table_plot = coef_table.sort_values("Coefficient")

plt.barh(coef_table_plot["Variable"], coef_table_plot["Coefficient"])
plt.xlabel("Logistic Regression Coefficient")
plt.title("Effect of Batting Variables on Winning Team Classification")
plt.show()
```



- Variables farther right help predict winning teams

- Variables farther left hurt that prediction or reflect overlap with stronger variables

16 Predicted Winning Probabilities

```
mlb[["Tm", "Win%", "winning_team", "predicted_win_probability"]].sort_values(
    "predicted_win_probability", ascending=False
)
```

	Tm	Win%	winning_team	predicted_win_probability
0	Arizona D'Backs	0.549	1	0.999968
18	New York Yankees	0.580	1	0.998398
13	Los Angeles Dodgers	0.605	1	0.996527
16	Minnesota Twins	0.506	1	0.990039
20	Philadelphia Phillies	0.586	1	0.979909
2	Baltimore Orioles	0.562	1	0.979583
17	New York Mets	0.549	1	0.979010
10	Houston Astros	0.547	1	0.978343
22	San Diego Padres	0.574	1	0.965035
4	Chicago Cubs	0.512	1	0.940085
15	Milwaukee Brewers	0.574	1	0.910322
3	Boston Red Sox	0.500	0	0.774570
1	Atlanta Braves	0.549	1	0.761785
7	Cleveland Guardians	0.571	1	0.748491
24	San Francisco Giants	0.494	0	0.711403
11	Kansas City Royals	0.531	1	0.523891
27	Texas Rangers	0.481	0	0.426355
25	St. Louis Cardinals	0.512	1	0.408520
28	Toronto Blue Jays	0.457	0	0.395487
8	Colorado Rockies	0.377	0	0.311322
9	Detroit Tigers	0.531	1	0.298858
6	Cincinnati Reds	0.475	0	0.297127
23	Seattle Mariners	0.525	1	0.288856
12	Los Angeles Angels	0.389	0	0.138879
29	Washington Nationals	0.438	0	0.071652
21	Pittsburgh Pirates	0.469	0	0.051402
19	Oakland Athletics	0.426	0	0.036301
14	Miami Marlins	0.383	0	0.019897
26	Tampa Bay Rays	0.494	0	0.017934
5	Chicago White Sox	0.253	0	0.000052

- Teams with the highest predicted probabilities should be strong offensive teams

- This helps compare actual team success to model-based expectations

---

17 K-Means Clustering Goal

- Group teams by **offensive style**
- Do this without using winning percentage
- See whether offensive styles line up with team success

---

18 Standardize Variables for Clustering

```
cluster_vars = mlb[["R", "HR", "BA", "RBI", "SB", "OBP", "SLG", "OPS"]].copy()

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_vars)
```

Why this matters:

- Clustering works better when variables are on the same scale

- Standardization prevents one stat from dominating the grouping

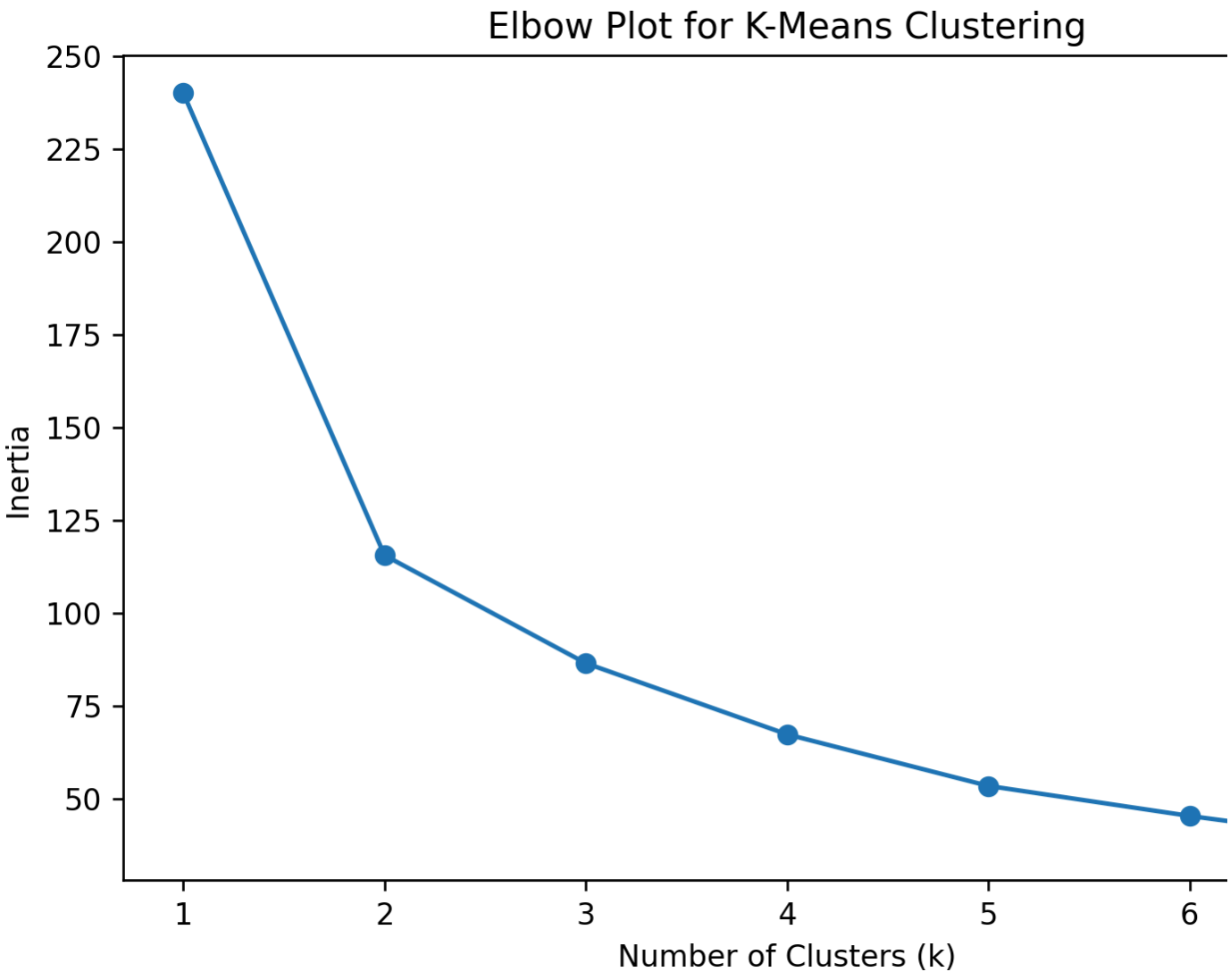
---

19 Choose Number of Clusters

```
inertia_values = []

for k in range(1, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    km.fit(cluster_scaled)
    inertia_values.append(km.inertia_)

plt.plot(range(1, 8), inertia_values, marker="o")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Plot for K-Means Clustering")
plt.show()
```



What this shows:
This elbow plot shows how the within-cluster variation (inertia) decreases as the number of clusters (k) increases. A sharp drop followed by a leveling off indicates the simplest and most efficient number of clusters.

Key insights:

- There is a large drop in inertia from k = 1 to k = 3, meaning adding clusters in this range significantly improves how well teams are grouped
- The “elbow” point appears around k = 3, suggesting that three clusters provide a good balance between simplicity and accuracy

What this means for the project:
Choosing 3 clusters allows us to group MLB teams into distinct offensive profiles without overcomplicating the model. This supports the goal of identifying meaningful patterns in team performance, showing that teams can be categorized into a few clear types based on their offensive statistics rather than needing many complex groupings.

20 Fit Final K-Means Model

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=20)
mlb["cluster"] = kmeans.fit_predict(cluster_scaled)

mlb[["Tm", "cluster"]].sort_values("cluster")
```

	Tm	cluster
1	Atlanta Braves	0
4	Chicago Cubs	0
7	Cleveland Guardians	0
8	Colorado Rockies	0
9	Detroit Tigers	0
11	Kansas City Royals	0
19	Oakland Athletics	0
16	Minnesota Twins	0
24	San Francisco Giants	0
25	St. Louis Cardinals	0
27	Texas Rangers	0
28	Toronto Blue Jays	0
13	Los Angeles Dodgers	1
15	Milwaukee Brewers	1
3	Boston Red Sox	1
0	Arizona D'Backs	1
22	San Diego Padres	1
20	Philadelphia Phillies	1
17	New York Mets	1
18	New York Yankees	1
10	Houston Astros	1
2	Baltimore Orioles	1
5	Chicago White Sox	2
6	Cincinnati Reds	2
14	Miami Marlins	2
12	Los Angeles Angels	2
23	Seattle Mariners	2

	Tm	cluster
21	Pittsburgh Pirates	2
26	Tampa Bay Rays	2
29	Washington Nationals	2

What this shows:
Each team has been assigned to one of three clusters based on their offensive statistics. These clusters group teams with similar hitting profiles.

What this gives us:

- Every team is now placed into a meaningful offensive category instead of just raw numbers
- We can directly compare these groups to winning percentage, `win%`, to see which offensive profiles lead to success
- This helps answer the main project question by showing not just *what stats matter*, but *what types of teams tend to win*

---

21 Cluster Summary Table

```
cluster_summary = mlb.groupby("cluster")[["R", "HR", "BA", "RBI", "SB", "OBP", "SLG", "OPS", "Win%
cluster_summary
```

	R	HR	BA	RBI	SB	OBP	SLG	OPS	Win%
cluster									
0	695.917	177.667	0.241	664.583	95.50	0.308	0.396	0.704	0.496
1	790.900	207.200	0.255	757.900	126.90	0.326	0.425	0.751	0.563
2	635.375	156.125	0.232	603.125	150.25	0.301	0.370	0.672	0.428

Main findings:

- **Cluster 1 represents the strongest offensive teams**
  → Highest averages in runs (790), HR (207), OBP (.326), SLG (.425), and OPS (.751)
  → Also has the highest Win% (~.563), showing a clear link between strong offense and winning
- **Cluster 2 represents the weakest offensive teams**
  → Lowest runs (635), HR (156), OBP (.301), SLG (.370), and OPS (.672)
  → Lowest Win% (~.428), indicating weaker offense is associated with losing teams
- **Cluster 0 represents average or middle-tier teams**
  → Moderate offensive stats across all categories
  → Win% (~.496) is close to .500, suggesting these teams are more balanced but less dominant

- **Clear pattern:**
  → As offensive production increases across clusters, winning percentage also increases
  → This shows a strong relationship between hitting performance and team success

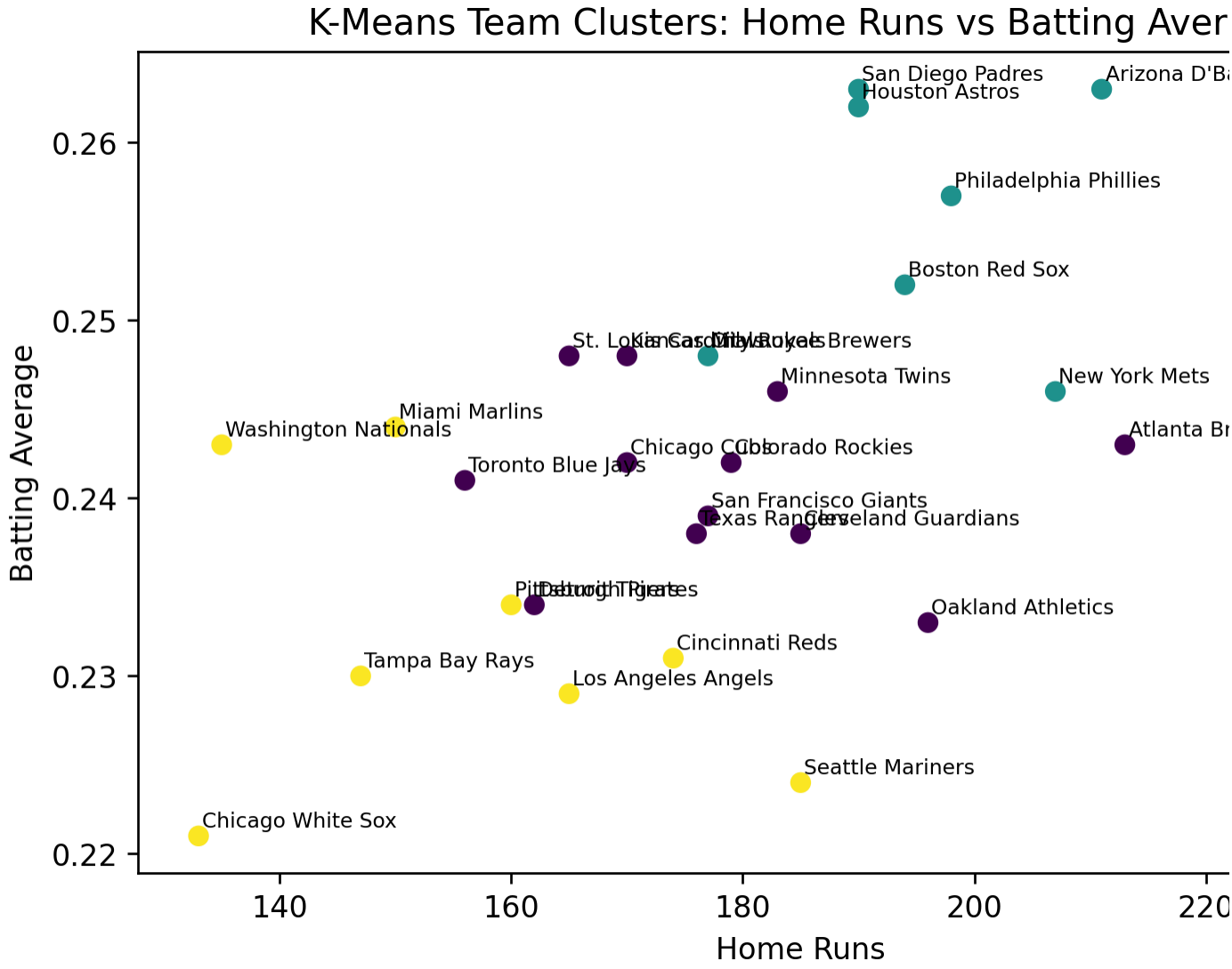
What this means:
The clustering confirms that teams naturally group into tiers based on offensive strength, and these tiers directly align with winning outcomes. This supports the main conclusion that better offensive performance leads to greater team success.

---

22 Cluster Plot: HR vs BA

```
plt.scatter(mlb["HR"], mlb["BA"], c=mlb["cluster"])
for _, row in mlb.iterrows():
    plt.text(row["HR"] + 0.3, row["BA"] + 0.0005, row["Tm"], fontsize=7)

plt.xlabel("Home Runs")
plt.ylabel("Batting Average")
plt.title("K-Means Team Clusters: Home Runs vs Batting Average")
plt.show()
```



Insight:

- This helps visualize how teams differ by power and hitting quality

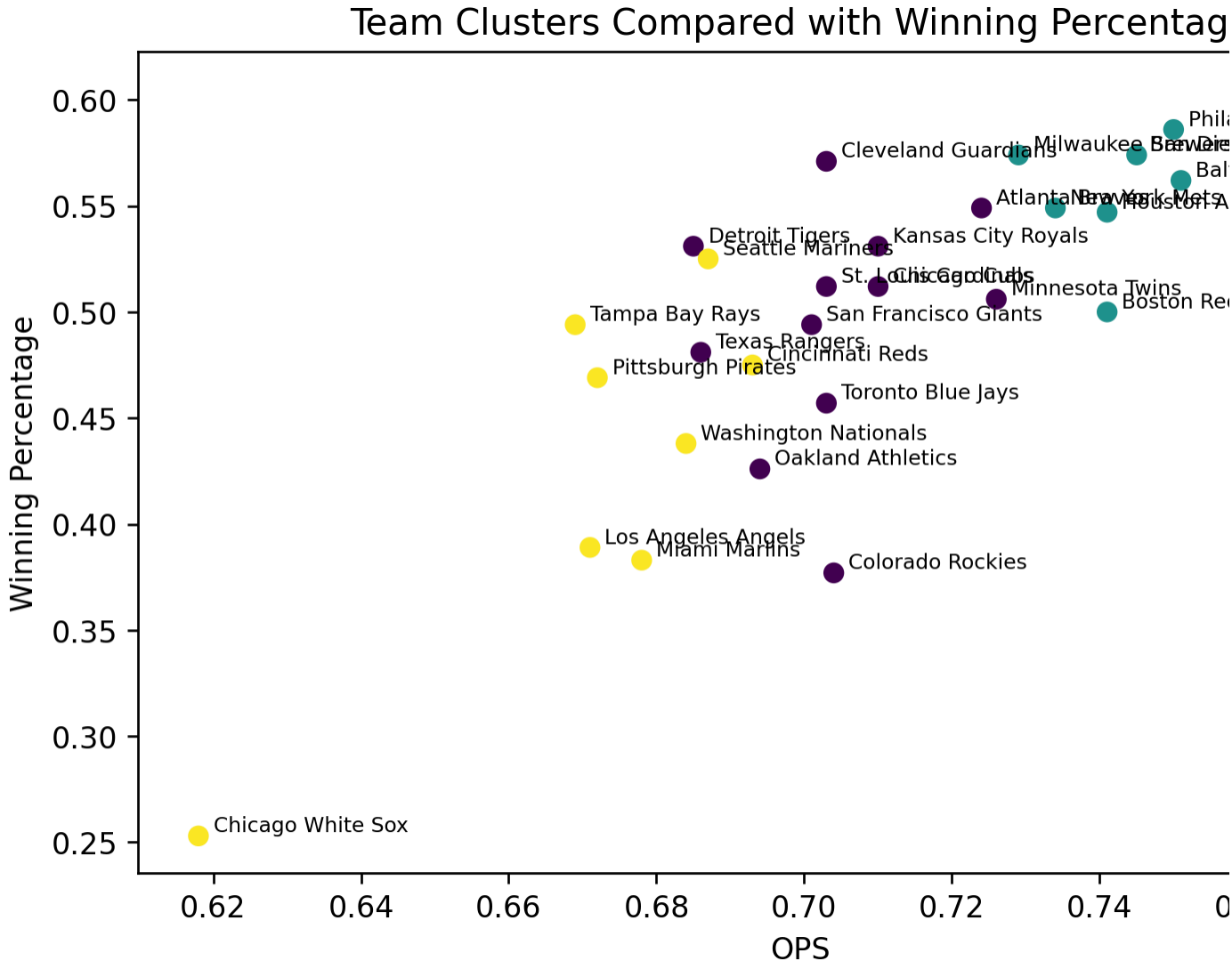
- Teams in the same color have similar offensive profiles

---

23 Cluster Plot: OPS vs Winning Percentage

```
plt.scatter(mlb["OPS"], mlb["Win%"], c=mlb["cluster"])
for _, row in mlb.iterrows():
    plt.text(row["OPS"] + 0.002, row["Win%"] + 0.002, row["Tm"], fontsize=7)

plt.xlabel("OPS")
plt.ylabel("Winning Percentage")
plt.title("Team Clusters Compared with Winning Percentage")
plt.show()
```



Main takeaway:

- The best offensive cluster should also have the best winning percentages

- This connects team style directly to team success

---

24 Cluster Success Summary

```
cluster_win_summary = mlb.groupby("cluster")[["Win%", "winning_team"]].agg(["mean", "sum", "count"
cluster_win_summary
```

	Win%			winning_team		
	mean	sum	count	mean	sum	count
cluster						
0	0.496	5.947	12	0.583	7	12
1	0.563	5.626	10	0.900	9	10
2	0.428	3.426	8	0.125	1	8

Key Takeaways:

- **Cluster 1 is the most successful group**
  → Highest Win% (~.563) and 9/10 teams are winning
- **Cluster 0 is average**
  → Around .500 Win% with about half the teams winning
- **Cluster 2 is the weakest group**
  → Lowest Win% (~.428) and only 1/8 teams are winning

What this means:
Teams with stronger offensive profiles are much more likely to win, while weaker offensive teams consistently perform worse.

---

25 Final Findings

25.1 1. Offense matters

- Runs, OPS, OBP, and SLG are strongly related to winning

25.2 2. Logistic regression works

- Batting stats alone do a good job of predicting winning teams

25.3 3. Teams fall into clear offensive groups

- Strong offense, average offense, and weak offense
- The strongest offensive cluster also performs best overall

---

26 Limitations

- This project only uses batting data
- It does not include:
  - pitching
  - defense

- injuries
- schedule difficulty

---

27 Conclusion

This project showed that offense matters a lot when it comes to winning in Major League Baseball. Teams that score more runs, hit more home runs, and post strong numbers in categories like OPS and OBP tend to win more games. The correlation analysis backed this up, finding clear connections between offensive stats and winning percentage.

The logistic regression model took things a step further by showing that batting stats can actually be used to predict whether a team will finish with a winning record. This means offensive numbers are not just tied to success — they can help forecast it.

The K-means clustering portion of the project grouped teams based on their offensive tendencies. Some teams are built around power, others put up well-rounded numbers across the board, and some simply struggle to produce at the plate. Looking at these groups made it easier to see how different approaches to hitting play out over the course of a season.

At the end of the day, this project is a good example of how data analysis can be applied to baseball to find patterns that might not be obvious at first glance. Using a mix of statistical tools and basic machine learning, it becomes possible to get a clearer sense of what actually drives winning and what separates good teams from bad ones.

---

28 References

- Hayes, A. (2025). *MLB Team Batting Statistics (2024).* SCORE Sports Data Repository. https://data.scorenetwork.org/baseball/mlb-team-batting-2024.html
- Sanders, S. (2025). *MLB Standings 2024.* SCORE Sports Data Repository. https://data.scorenetwork.org/baseball/mlb-standings-2024.html
- Baseball-Reference. (2024). *2024 Major League Baseball season statistics.* https://www.baseball-reference.com/leagues/majors/2024.shtml